Assignment 6: Medians and Order Statistics & Elementary Data Structures

MSCS-532: Algorithms and Data Structures

University of the Cumberlands

Introduction

This report examines order-statistic selection and elementary data structures through Python implementations, complexity analysis, and empirical testing. Part 1 compares deterministic Median of Medians selection with Randomized Quickselect. Part 2 evaluates array and matrix operations, an array-backed stack, a dynamically resized circular-array queue, and a singly linked list. The focus is on correctness, trade-offs, and practical uses rather than execution speed alone (Cormen et al., 2022).

Part 1: Selection Algorithms

Implementation and Design

Both selection functions return the kth smallest element using one-based k indexing. They reject empty input and invalid k values, use three-way partitions (smaller, equal, and larger than the pivot), and leave the caller's list unchanged. Three-way partitioning avoids unnecessary recursion when multiple values equal the pivot. The implementations allocate new lists during partitioning, a design choice that favors clarity and preservation of input over in-place memory efficiency.

Median of Medians divides the input into groups of at most five, sorts each small group with insertion sort, and recursively selects a pivot from the group medians. This pivot limits how unbalanced a partition can become. Randomized Quickselect instead chooses a uniformly random element from the current list as pivot, then recursively searches only the partition containing the requested rank. The randomized approach reduces pivot-selection overhead but does not guarantee a balanced split on every run.

Theoretical Performance Analysis

For deterministic selection, grouping into fives takes O(n) work. At least a constant fraction of elements can be excluded from further selection after partitioning. A standard upper-bound recurrence is $T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + O(n)$, which solves to O(n) worst-case time. The fixed-size insertion sorts are O(1) per group, hence O(n) collectively (Cormen et al., 2022).

For Randomized Quickselect, partitioning costs O(n), while the size of the recursive subproblem depends on the randomly chosen pivot. Its expected recurrence can be bounded by $E[T(n)] \leq (1/n) \Sigma_{i=1}^{n} E[T(\max(i - 1, n - i))] + O(n)$, yielding expected O(n) time. Repeatedly choosing extreme pivots produces the $O(n^2)$ worst case. Randomization makes that unfavorable sequence unlikely, not impossible.

Both implementations create additional lists during partitioning. The total auxiliary space used by these partition-based implementations is O(n), including retained partition lists along an active recursion path. In the worst case, Randomized Quickselect may also require O(n) recursive stack frames when pivots repeatedly produce highly unbalanced partitions. The deterministic algorithm additionally creates groups and lists of medians. These overheads should be distinguished from those of in-place selection implementations.

Empirical Method

Benchmarks used input sizes of 100, 500, 1,000, and 5,000, with random, sorted, reverse-sorted, and duplicate-heavy distributions. For each configuration, both algorithms received the same dataset and selected the middle-ranked value, $k = (n + 1) // 2$. Each algorithm was timed seven times using Python's time.perf_counter(), and the median execution time was reported in milliseconds. A sorted reference checked correctness before each measurement was accepted.

Dataset generation used a fixed pseudorandom seed of 532; however, the randomized algorithm's pivot selections were not separately seeded. These measurements represent this execution rather than guaranteed performance on other machines.

Empirical Results

n	Distribution	Deterministic (ms)	Randomized (ms)
100	Random	0.1302	0.0275
100	Sorted	0.0951	0.0248
100	Reverse sorted	0.2206	0.0220
100	Duplicate heavy	0.1590	0.0337
500	Random	1.0168	0.2929
500	Sorted	0.7915	0.1812
500	Reverse sorted	0.6115	0.2096
500	Duplicate heavy	0.4995	0.0939
1,000	Random	0.8269	0.2345
1,000	Sorted	1.0745	0.1614
1,000	Reverse sorted	1.5307	0.3299
1,000	Duplicate heavy	0.3722	0.1548
5,000	Random	5.0332	0.6468
5,000	Sorted	3.5505	0.6873
5,000	Reverse sorted	5.6821	0.7080
5,000	Duplicate heavy	2.2598	1.0687

Randomized Quickselect was faster in all 16 observed configurations. For 5,000 random elements, its median time was 0.6468 ms versus 5.0332 ms for Median of Medians. This difference is consistent with the deterministic algorithm's additional work selecting a guaranteed-quality pivot. Duplicate-heavy inputs did not cause incorrect results because equal values were grouped together. Timing does not increase monotonically for every input size: the measurements are short, and interpreter overhead, pivot choices, and system variability can influence individual results. Although Randomized Quickselect performed faster in every tested configuration, these results should not be generalized to all datasets or execution environments. The measurements illustrate practical overhead rather than proving asymptotic bounds.

Part 2: Elementary Data Structures

Implementation and Operation Costs

ArrayOperations uses a Python list and explicitly shifts elements during insertion and deletion. MatrixOperations uses a rectangular list of row lists, validates column counts, and supports row insertion, row deletion, and indexed access. ArrayStack uses append and pop operations at the end of a list. ArrayQueue uses a circular array, a front index, an element count, and capacity doubling. It copies elements in their logical queue order when resizing. SinglyLinkedList maintains head, tail, and size references, supporting insertion at the beginning, end, or a specified position; deletion by value or position; searching; and traversal.

Structure / operation	Time complexity	Explanation
Array access	O(1)	Direct index
Array insert/delete at position	O(n) worst case	Elements shifted
Matrix cell access	O(1)	Two direct indexes
Matrix row insert	O(r + c) worst case	Copy new row and shift row references
Matrix row delete	O(r) worst case	Shift row references
Stack push	O(1) amortized	Occasional list resize
Stack pop/peek	O(1)	End-of-array operation
Circular queue enqueue	O(1) amortized	Occasional capacity doubling
Circular queue dequeue/front	O(1)	No element shifting
Linked list insert at head/tail	O(1)	Head/tail references maintained
Linked list search/delete by value	O(n)	Traversal required
Linked list traverse	O(n)	Visit each node

Here r is the number of matrix rows and c is the number of columns. Matrix row operations shift references rather than copying all cell values; row insertion additionally constructs a new row of c values. These bounds describe the provided Python implementations rather than every possible matrix representation.

Trade-offs and Applications

Arrays provide fast indexed access and good locality, making them useful for lookup-heavy workloads and matrix-like tabular data. Their disadvantage is the cost of inserting or deleting elements near the front. Singly linked lists permit constant-time insertion at the head and, with a tail reference, at the end, but finding an arbitrary position requires traversal and each

node carries reference overhead. For stacks, an array-backed design is straightforward and efficient because operations occur at one end. For queues, a circular array avoids the O(n) shifting that would result from repeatedly deleting the first list element; a linked-list queue with both head and tail references is another viable O(1) enqueue/dequeue design.

Examples include a stack for undo history or expression evaluation, a queue for first-in-first-out task scheduling, a linked list for sequences requiring frequent head insertions, and a matrix for structured numeric measurements. The best structure depends on access patterns, memory limits, resizing behavior, and ease of implementation rather than one universal performance ranking (Cormen et al., 2022).

Testing, Limitations, and Reproducibility

The selection test program reported 208 passing checks across both algorithms, including valid ranks, invalid ranks, duplicate values, and varied input distributions. The data-structure test program reported 66 passing checks for arrays, matrices, stacks, queues, and singly linked lists, including empty structures, invalid indexes, queue wraparound and resizing, and linked-list head/tail updates. These are successful tests of the supplied cases, not formal proof of correctness. Benchmarking was limited to four input sizes, four distributions, one requested rank per dataset, and one execution environment. Future work could test more rank positions, much larger inputs, memory allocation, and repeated independent benchmark sessions.

Conclusion

Median of Medians provides a worst-case linear-time guarantee at the cost of additional pivot-selection work, while Randomized Quickselect achieved lower measured times in this experiment with expected linear-time behavior. The elementary data structures demonstrate complementary strengths: arrays offer indexed access, stacks and circular queues efficiently

manage restricted access orders, and linked lists support flexible node insertion. Correctness tests, careful complexity statements, and workload-specific selection are essential to defensible algorithm and data-structure design.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.